via the `Recipe` entity's `Ingredients` property to do so, as you'll see in section 12.4.

For this simple example, your data model consists of these three classes: `AppDbContext`, `Recipe`, and `Ingredient`. The two entities are mapped to tables and their columns to properties, and you use the `AppDbContext` to access them.

NOTE This *code-first* approach is typical, but if you have an existing database, you can automatically generate the EF entities and `DbContext` instead. (You can find more information in Microsoft's "reverse engineering" article at http://mng.bz/mgd4.)

The data model is complete, but you're not quite ready to use it: your ASP.NET Core app doesn't know how to create your `AppDbContext`, and your `AppDbContext` needs a connection string so that it can talk to the database. In section 12.2.3 we tackle both of these problems, and we finish setting up EF Core in your ASP.NET Core app.

12.2.3 Registering a data context

As with any other service in ASP.Net Core, you should register your `AppDbContext` with the dependency injection (DI) container. When registering your context, you also configure the database provider and set the connection string so that EF Core knows how to talk with the database.

You register the `AppDbContext` with the `WebApplicationBuilder` in Program.cs. EF Core provides a generic `AddDbContext<T>` extension method for this purpose; the method takes a configuration function for a